

PreOne Enterprise Security Architecture

ADR-113 — Enterprise Preschool Operating System

Document Version: 1.0

Status: LOCKED

ADR Reference: ADR-113

Date: 2026-07-14

Product: PreOne — Enterprise Preschool Operating System

Predecessor: Vision v1.0, BRC v1.0, Master PRD v1.0, DDD v1.0, ERD v3.0, API Contract Catalog v1.0, OpenAPI Spec v1.0, ADR-111 DevOps v1.0

Successor: DevOps Runbook, SRE Playbook, Incident Response Plan, Penetration Test Report, DPDP Compliance Pack

Scope: Zero Trust + RBAC + Encryption + Audit + DPDP + Key Management + API Security + Monitoring + Incident Response

Classification: Internal Engineering Reference

Prepared by: PreOne Architecture & Engineering Team

1. Security Architecture Overview.....	1
1.1 End-to-End Request Flow.....	1
1.2 Stage-by-Stage Controls.....	4
1.3 Trust Boundaries.....	4
2. Security Principles.....	5
2.1 Principle Precedence.....	5
2.2 Principle Enforcement in CI.....	5
3. Threat Model.....	6
3.1 External Threats.....	6
3.2 Internal Threats.....	6
3.3 Cloud Threats.....	7
3.4 Threat Modeling Process.....	7
4. OWASP Top 10 Compliance.....	7
4.1 Compliance Validation Cadence.....	8
4.2 False Positive Triage.....	8
5. Authentication.....	8
5.1 Supported Authentication Methods.....	9
5.2 Password Policy.....	9
5.3 Session Management.....	9
5.3.1 Device Recognition.....	10
5.3.2 Anomaly Detection.....	10
6. JWT Architecture.....	10
6.1 Token Issuance Flow.....	10
6.2 JWT Claims.....	13
6.3 JWT Security Controls.....	14
6.4 Key Management for JWT Signing.....	14
6.5 Token Revocation.....	14

7. Multi-Factor Authentication (MFA)	15
7.1 Mandatory MFA Roles	15
7.2 Supported MFA Factors	15
7.3 MFA Enforcement	16
7.4 MFA Bypass & Recovery	16
8. Authorization	16
8.1 Authorization Flow	16
8.2 Authorization Features	20
8.3 Permission Bundle Architecture	20
8.4 Field-Level Security	20
8.5 Approval Workflow	20
9. Encryption	21
9.1 Encryption in Transit	21
9.2 Encryption at Rest	21
9.3 Sensitive Data Classes	22
9.4 Key Hierarchy	22
9.5 Cryptographic Standards	22
10. Object Storage Security	23
10.1 Storage Security Controls	23
10.2 Upload Workflow	23
10.3 Download Workflow	23
10.4 Tenant Isolation	24
10.5 Lifecycle and Retention	24
11. Audit Trail	24
11.1 Audited Actions	24
11.2 Audit Record Schema	25
11.3 Hash Chain Integrity	25
11.4 Audit Log Retention & Access	25

11.5 Audit Query API.....	26
12. DPDP Compliance (India).....	26
12.1 Key DPDP Controls.....	26
12.2 Consent Capture.....	27
12.3 Children's Data.....	27
12.4 Data Principal Rights.....	27
12.5 Breach Notification.....	27
13. Key Management.....	28
13.1 Secret Stores.....	28
13.2 Managed Keys.....	28
13.3 Key Rotation Process.....	29
13.4 Emergency Key Revocation.....	29
13.5 Secret Access Audit.....	29
14. API Security.....	29
14.1 API Security Controls.....	30
14.2 Rate Limiting Strategy.....	30
14.3 Idempotency.....	30
14.4 API Versioning.....	31
14.5 GraphQL (Future).....	31
15. Security Monitoring.....	31
15.1 Monitored Signals.....	31
15.2 Alert Severities.....	32
15.3 SIEM Correlation Rules.....	32
15.4 Security Dashboard.....	32
15.5 Threat Intelligence.....	33
16. Incident Response.....	33
16.1 Incident Response Phases.....	33
16.2 Incident Roles.....	33

16.3 Incident Severity Classification.....	34
16.4 Post-Incident Review.....	34
16.5 Incident Communication.....	34
17. Security Testing.....	35
17.1 Security Test Suite.....	35
17.2 Remediation SLAs.....	35
17.3 Penetration Testing Cadence.....	35
17.4 Bug Bounty (Future).....	36
18. Security Headers.....	36
18.1 Security Headers Reference.....	36
18.2 Content-Security-Policy Deep Dive.....	37
18.3 Header Validation.....	37
19. Final Security Stack.....	37
19.1 Stack Validation.....	37
19.2 Stack Evolution.....	38
20. Cross References.....	38
20.1 Document Control.....	38
21. Glossary.....	39

1. Security Architecture Overview

PreOne Enterprise Preschool Operating System processes sensitive personal data of children, parents, and staff across hundreds of schools in India. The security architecture is designed as a defense-in-depth stack where every request traverses ten independent security layers — from the user's device, through edge encryption, web application firewall, authentication, authorization, application logic, audit, and finally to encrypted data stores. Each layer enforces its own controls independently, so compromise of any single layer does not grant access to protected data.

The architecture is grounded in Zero Trust: no implicit trust is granted based on network location, prior authentication, or service identity. Every API request must present a valid JWT, must pass scope validation, must be authorized at the route and service layer, and must be audited. Internal service-to-service calls are similarly authenticated via mTLS through the service mesh. This document defines the controls, technologies, and operational practices that constitute the PreOne security baseline.

1.1 End-to-End Request Flow

The flow diagram below illustrates the path of a typical authenticated request from a parent's mobile device to the encrypted data stores. Each stage applies its own security control and is independently testable, observable, and enforceable. The diagram should be read top-down — a request that fails any stage is rejected with a 4xx response and never reaches the next layer.

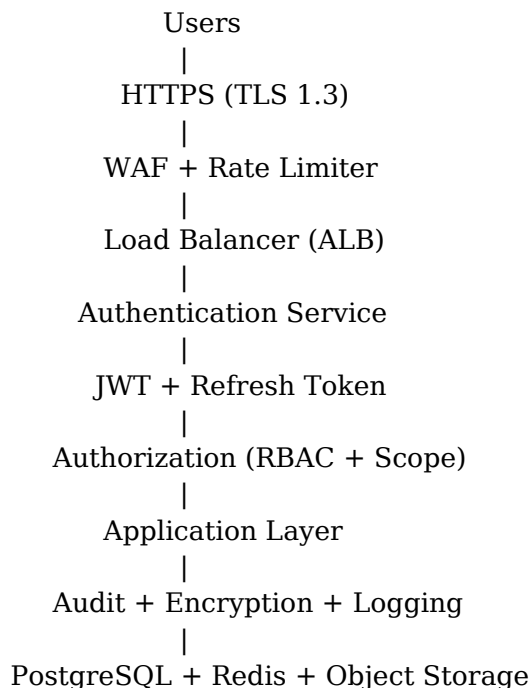


Figure 1.1 — Security architecture: 10-layer defense-in-depth request flow

1.2 Stage-by-Stage Controls

The table below enumerates each stage of the request flow, the security control applied at that stage, and the rationale. Engineers referencing this table should treat each stage as a non-bypassable checkpoint — short-circuiting any stage is a Critical-severity security defect.

#	Layer	Control	Description
1	Users	End users on Mobile App / Web App / Parent Portal	External entry point. Devices are untrusted; all client input must be validated server-side.
2	HTTPS (TLS 1.3)	Encrypted transport, HSTS, modern cipher suites	TLS 1.3 enforced with HSTS preload; TLS 1.0/1.1 disabled; perfect forward secrecy mandatory.
3	WAF + Rate Limiter	AWS WAF + NGINX rate limit + bot rules	Layer 7 firewall blocks SQLi, XSS, SSRF, known-bad IPs; rate limiter caps requests per IP and per token.
4	Load Balancer (ALB)	AWS Application Load Balancer + health checks	Distributes traffic across Kubernetes worker nodes; performs TLS termination, path-based routing, health gating.
5	Authentication Service	PreOne Auth Service (NestJS) + OTP gateway	Issues JWT access + refresh tokens; verifies OTP via SMS/email; enforces password policy and logout.
6	JWT + Refresh Token	RS256 signed access (15m) + refresh (30d, rotated)	Short-lived access token limits exposure; refresh token rotated on every use; revocation list maintained in Redis.

#	Layer	Control	Description
7	Authorization (RBAC + Scope)	Permission bundles + tenant/school/branch scope	Each request is checked against user role, permission version, and data scope; field-level security applied.
8	Application Layer	NestJS guards + Prisma middleware + DTO validation	Defense in depth: route guard, controller-level scope check, service-level ownership check, Prisma row filter.
9	Audit + Encryption + Logging	Immutable audit trail + AES-256 + structured logs	Every sensitive action is audited; PII encrypted at column level; logs shipped to Loki without sensitive payloads.
10	PostgreSQL + Redis + Object Storage	Encrypted volumes + network isolation + signed URLs	DB in private subnet; Redis AUTH + TLS; S3/MinIO with SSE-KMS and pre-signed time-bound URLs.

1.3 Trust Boundaries

The architecture defines four explicit trust boundaries. Each boundary crossing requires re-authentication, re-authorization, or cryptographic verification. Boundary 1 is the network edge — untrusted Internet meets PreOne's authenticated HTTPS endpoint. Boundary 2 is the application perimeter — WAF/ALB meets the authenticated API namespace. Boundary 3 is the service boundary — microservices communicate only via mTLS through the service mesh. Boundary 4 is the data boundary — application pods reach PostgreSQL, Redis, and object storage only through scoped IAM roles and private subnets. No component is permitted to operate across more than one boundary without explicit security review.

This boundary discipline means, for example, that a compromised application pod still cannot read database backups (different boundary, different IAM role, different KMS key), and a leaked database credential cannot be used from outside the VPC (private subnet, security group restriction). The boundaries are tested quarterly via red-team exercises.

2. Security Principles

Eight principles govern every security decision in the PreOne platform. These principles are referenced in code reviews, architecture reviews, threat models, and incident postmortems. When two principles appear to conflict (e.g., Least Privilege vs. operational efficiency), the more restrictive principle wins by default; deviations require explicit ADR approval and security team sign-off.

Each principle below is accompanied by its concrete implementation in the PreOne stack. Engineers are expected to internalize these principles and apply them in day-to-day design decisions — not only when writing security-sensitive code, but also when adding features, refactoring, or fixing bugs. A pull request that violates a principle will be blocked by the security reviewer even if the immediate change appears harmless.

Principle	Description & Implementation
Zero Trust Architecture	Never trust, always verify. Every request — internal or external — is authenticated, authorized, and encrypted. No implicit trust based on network location.
Least Privilege Access	Users, services, and components receive only the minimum permissions required to perform their function. Roles are scoped tightly; super-admin actions require MFA.
Defense in Depth	Multiple independent security layers (WAF, ALB, Auth, RBAC, ORM, DB encryption) so a single layer compromise does not expose the system.
Secure by Default	All features ship with the most secure configuration enabled. Insecure options must be explicitly opted into and reviewed during code review.

Principle	Description & Implementation
Privacy by Design	Data minimization, purpose limitation, and consent are baked into the architecture — not bolted on after launch. PII collection requires explicit parent consent.
Fail Secure	When a component fails — auth service down, cache miss, token validation error — the system defaults to deny access rather than allow.
Complete Auditability	Every state-changing or sensitive action is recorded with user, role, IP, device, before/after values, and timestamp. Audit logs are immutable and append-only.
DPDP Compliance	Architecture aligns with India's Digital Personal Data Protection Act — explicit consent, data residency in India, breach notification workflow, right to correction and erasure.

2.1 Principle Precedence

When principles conflict, the following precedence applies (highest to lowest): Fail Secure > Privacy by Design > Least Privilege > Zero Trust > Defense in Depth > Complete Auditability > Secure by Default > DPDP Compliance. For example, if Fail Secure (deny on error) conflicts with operational efficiency, the system must deny. If Privacy by Design (data minimization) conflicts with feature completeness, the feature ships with less data collection. This precedence is encoded in the Security Decision Matrix maintained by the architecture team.

2.2 Principle Enforcement in CI

Several principles have automated enforcement in CI: (1) Defense in Depth — PRs that remove a security layer (e.g., disable WAF rule, skip audit log) require security team approval. (2) Complete Auditability — a linter flags new state-changing routes without @Audit decorator. (3) Secure by Default — IaC templates with insecure defaults (e.g., public S3 bucket) fail the policy-as-code check. (4) Least Privilege — IAM role changes

trigger an automated over-permissioning scan. Principles without automated checks are enforced via mandatory security review on every ADR and high-risk PR.

3. Threat Model

PreOne's threat model enumerates the adversaries, their capabilities, and the controls that mitigate each threat. The model is reviewed quarterly and after every major incident. Threats are categorized into three groups: External Threats (originating outside PreOne — the Internet, partner networks), Internal Threats (originating inside PreOne — employees, contractors, compromised internal accounts), and Cloud Threats (originating from misconfiguration or compromise of cloud infrastructure). Each threat lists a concrete mitigation; threats without a concrete mitigation are tracked as security work items with an SLA.

3.1 External Threats

External threats target the public-facing surface: login endpoints, file upload, public APIs, and the web application. These threats are highest-volume and best-mitigated by automated controls (WAF, rate limiting, input validation) supplemented by anomaly detection. The mitigation column lists the primary control; defense-in-depth means each threat is also covered by secondary controls listed in the relevant chapter.

Threat	Mitigation
Credential Stuffing	Rate limiting + breached-password check (HaveIBeenPwned API) + account lockout after 5 failed attempts + bot detection on login endpoint.
Brute Force Login	Exponential backoff per IP and per account; CAPTCHA after 3 failures; MFA enforced for privileged accounts.
DDoS Attacks	AWS Shield Standard on edge + CloudFront distribution + WAF rate rules + auto-scaling K8s cluster + circuit breakers on downstream calls.
SQL Injection	Prisma ORM with parameterized queries everywhere; no raw SQL string concatenation; input validation via class-validator DTOs.

Threat	Mitigation
Cross-Site Scripting (XSS)	React escapes by default; CSP header with strict nonce; no dangerouslySetInnerHTML; HttpOnly + Secure + SameSite=Strict cookies.
CSRF	JWT in Authorization header (not cookies) eliminates CSRF surface; double-submit token for cookie-based routes; SameSite=Strict on session cookies.
Session Hijacking	Short-lived access tokens (15 min); refresh token rotation; device fingerprinting; IP+UA binding on token; revocation on anomaly.
API Abuse	Per-token rate limit (1000 req/min default, 100 req/min sensitive); per-IP rate limit; quota per tenant; spike detection alerts.
File Upload Malware	Magic-byte verification + MIME whitelist + ClamAV scan + size cap (10MB) + filename sanitization + isolated upload bucket.
Bot Attacks	AWS WAF bot rules + CAPTCHA challenge on suspicious patterns + device fingerprint + behavioral analytics on login + signup endpoints.

3.2 Internal Threats

Internal threats originate from authenticated users with some level of legitimate access — staff, contractors, or compromised internal accounts. These threats are harder to detect because the actor has valid credentials; mitigation relies on least-privilege RBAC, audit logging, anomaly detection, and segregation of duties. Insider threat mitigation also includes administrative controls: background checks, NDAs, access reviews, and break-glass workflows for sensitive operations.

Threat	Mitigation
Privilege Escalation	RBAC enforced at route guard + service layer + Prisma query scope; permission cache invalidated on role change; super-admin actions audited and MFA-gated.

Threat	Mitigation
Unauthorized Data Access	Row-level security in PostgreSQL per tenant; service accounts scoped to single namespace; DB access via bastion only; query audit logging.
Insider Threats	Just-in-time access via break-glass workflow; all admin actions recorded; quarterly access review; segregation of duties for sensitive operations.
Misconfigured Permissions	IaC-driven permissions (Terraform + K8s RBAC); config drift detection; CIS benchmark scan; automated policy validation in CI.
Data Leakage	DLP rules on email + Slack egress; USB drive restrictions on managed laptops; download audit trail; watermarking on exports of PII.
Accidental Deletion	Soft-delete on all entities; 30-day recycle bin; daily snapshots; PITR on PostgreSQL; delete actions require confirmation + audit.

3.3 Cloud Threats

Cloud threats arise from misconfiguration of AWS, Kubernetes, or third-party SaaS. These threats often lead to large-scale data exposure because cloud misconfigurations tend to affect entire tenants or the whole platform. Mitigation is primarily preventive: IaC with policy-as-code, CIS benchmarks, automated drift detection, and least-privilege IAM. Detective controls include CloudTrail, GuardDuty, and config-change alerts.

Threat	Mitigation
Secret Leakage	Secrets never in git (pre-commit hook + git-secrets); AWS Secrets Manager as source of truth; K8s Secrets mounted at runtime only; secret rotation automated.
Storage Misconfiguration	S3/MinIO buckets private by default; public access block on account level; bucket policy validation in CI; signed URLs with short TTL.

Threat	Mitigation
Kubernetes Misconfiguration	Pod Security Standards (restricted); network policies default-deny; RBAC reviewed quarterly; kube-bench CIS scan in CI; admission controllers (OPA Gatekeeper).
Database Exposure	PostgreSQL in private subnet only; no public RDS endpoint; access via bastion + IAM Auth; security group allow-lists application namespace only.
Backup Theft	Backups encrypted with separate KMS key; cross-region replication with KMS; backup access restricted to break-glass role; restore drills quarterly.

3.4 Threat Modeling Process

Threat modeling is performed for every new ADR and every major feature. The team uses the STRIDE methodology (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) applied to a data-flow diagram. The output is a threat register with severity, owner, and mitigation SLA. The register is reviewed monthly by the security engineering team and quarterly by the architecture review board. Threats rated High or Critical must be mitigated before the feature ships.

4. OWASP Top 10 Compliance

PreOne maps every OWASP Top 10 (2021) risk to a concrete architectural control. The mapping below is the authoritative reference — each row links an OWASP risk to its mitigation, the chapter in this ADR where the control is detailed, and the engineering owner. OWASP compliance is validated quarterly via external penetration testing and continuously via automated SAST/DAST scans in CI.

Compliance is not a one-time achievement but a continuous practice. Each OWASP risk has a named engineering owner accountable for the control's effectiveness, an automated test that validates the control in CI, and a metric tracked on the security dashboard (e.g., for A01 Broken Access Control, the metric is unauthorized-access attempt rate from DAST scans). Risk owners report monthly to the security steering committee.

OWASP Risk	Control	Mitigation Detail
A01 Broken Access Control	RBAC + Scope Validation	Route guards + service-level ownership check + Prisma row filter + field-level security; deny by default; integration tests cover privilege boundary.
A02 Cryptographic Failures	AES-256 + TLS 1.3	TLS 1.3 in transit; AES-256-GCM at rest; column-level encryption for PII; RSA-2048 / Ed25519 for JWT signing; no MD5/SHA1 anywhere.
A03 Injection	Prisma ORM + Parameterized Queries	All DB access via Prisma; parameterized queries only; no string interpolation in SQL; input validation via class-validator on every DTO.
A04 Insecure Design	DDD + Secure Review	Threat modeling in DDD; security design review before each ADR; abuse-case user stories; security acceptance criteria in PRD.
A05 Security Misconfiguration	Hardened Infrastructure	IaC with Terraform; CIS-hardened AMIs; K8s Pod Security Standards (restricted); automated config drift detection; no default credentials.
A06 Vulnerable Components	Dependency Scanning	Snyk + npm audit + Dependabot in CI; weekly patch cycle; CVE alerting on Slack; SBOM (CycloneDX) generated per release.
A07 Authentication Failures	JWT + MFA	Short-lived JWT (15 min); refresh rotation; MFA for privileged roles; account lockout; password breach check; session anomaly detection.

OWASP Risk	Control	Mitigation Detail
A08 Software & Data Integrity	Signed CI/CD Pipeline	Signed commits required; signed container images (cosign); immutable build artifacts; provenance attestation (SLSA Level 3).
A09 Logging & Monitoring Failures	Immutable Audit Logs	Append-only audit trail in PostgreSQL; logs shipped to Loki; Prometheus alerts on auth anomalies; SIEM correlation rules; 1-year retention.
A10 SSRF	Network Isolation	Outbound calls restricted by egress firewall; metadata endpoint blocked; allow-list of outbound domains; URL validation on user-supplied URLs.

4.1 Compliance Validation Cadence

OWASP compliance is validated on three cadences. Daily: SAST + dependency scan + secret scan in CI on every PR. Weekly: DAST scan on staging environment with full crawl. Quarterly: external penetration test by certified third party covering all OWASP risks + business-logic attacks. Annually: full OWASP compliance audit with attestation report shared with enterprise customers. Findings from any cadence are tracked in the security backlog with severity-based SLAs.

4.2 False Positive Triage

SAST and DAST tools generate noise — PreOne targets a false-positive rate below 20% via custom rule tuning and suppression rules. Every suppressed finding requires a written justification, an owner, and a re-validation date. Suppression rules are reviewed quarterly to ensure they do not mask genuine regressions. The security team maintains a finding-lifecycle dashboard: new -> triaged -> in-fix -> verified-suppressed -> closed.

5. Authentication

Authentication establishes the identity of the caller. PreOne supports multiple authentication methods to balance security with usability across user types — parents prefer OTP, back-office staff prefer password, and privileged roles require step-up MFA. All methods converge on a JWT-based session architecture described in Chapter 6. Authentication events (login, logout, failed login, MFA challenge) are audited and feed the anomaly-detection pipeline described in Chapter 15.

5.1 Supported Authentication Methods

The methods below are listed in order of preference. OTP is preferred for parents because it avoids password fatigue and leverages India's high mobile-phone penetration. Email + password is preferred for back-office users who need to switch devices less often and benefit from password-manager integration. All methods support device recognition and session management.

Method	Description
Mobile OTP Login	Primary method for parents and staff. Phone number + OTP via SMS gateway (MSG91). OTP valid for 5 minutes, 3 attempts max, resend after 60s cooldown.
Email + Password	Secondary method for back-office users. Password policy enforced (12+ chars, complexity, history). BCrypt/Argon2 hashing with per-user salt.
Refresh Tokens	Long-lived (30 days) rotation tokens stored hashed in DB. Single-use — each refresh issues a new token and invalidates the prior. Detects token theft via reuse.
Device Recognition	On first login from a new device, fingerprint (UA + screen + IP/24) is recorded. Subsequent logins from unrecognized device trigger OTP challenge even with valid refresh token.
Session Management	Active sessions listed in user profile; user can revoke any session; admin can force-logout a user; session count capped at 5 concurrent per user.

5.2 Password Policy

When passwords are used, the policy below is enforced server-side. Client-side validation is a UX nicety; server-side is authoritative. The policy is informed by NIST SP 800-63B — length is prioritized over complexity, breached passwords are rejected, and rate limiting is applied per account. Argon2id is the preferred hashing algorithm because it is memory-hard and resistant to GPU/ASIC brute-force attacks.

Minimum Length	12 characters
Maximum Length	128 characters (prevents DoS on hash)
Complexity	At least 1 uppercase, 1 lowercase, 1 digit, 1 special character
Password History	Last 5 passwords cannot be reused
Hashing Algorithm	Argon2id (preferred) with m=64MB, t=3, p=2 — fallback BCrypt cost 12
Breach Check	Password checked against HaveIBeenPwned k-anonymity API at signup and change
Account Lockout	5 consecutive failures → 15-min lock; 15 failures → 1-hour lock; 30 failures → admin unlock required
Reset Token TTL	15 minutes; single-use; invalidates all existing sessions on reset
Reset Channel	Email link + SMS OTP dual confirmation for privileged roles

5.3 Session Management

Session lifecycle is tightly controlled. A user may have up to five concurrent sessions (one per device class: phone, tablet, laptop, etc.). On the sixth login, the oldest session is revoked. Sessions are listed in the user's profile with device, IP, location, and last-active timestamp; the user can revoke any session. Administrators can force-logout a user (e.g., on role change) — this invalidates all refresh tokens for that user via permissionsVersion bump. On password reset, all existing sessions are revoked by default.

5.3.1 Device Recognition

On first login from a new device, a fingerprint is computed from user-agent, screen resolution, timezone, and IP/24 (to tolerate carrier-grade NAT). The fingerprint is hashed and stored server-side. On subsequent logins from a recognized device, the user proceeds directly. On an unrecognized device, an additional OTP challenge is sent even if the user provided valid credentials. This blocks credential-stuffing attacks where the attacker has the password but not the device.

5.3.2 Anomaly Detection

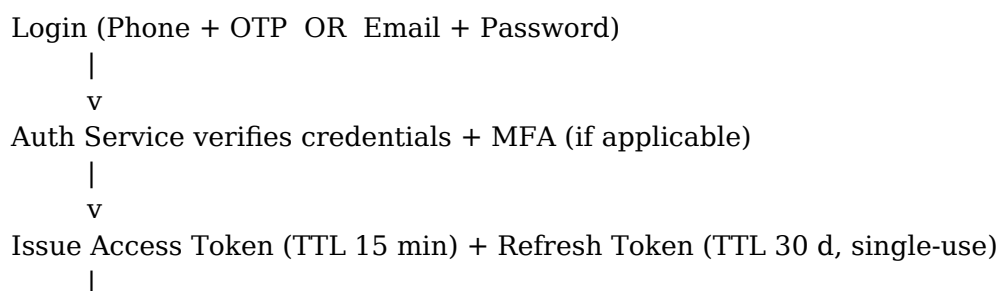
Login anomalies feed the security monitoring pipeline. Detected anomalies include: impossible travel (login from Mumbai then Delhi within 30 min), new-geo login (first login from a new state/country), new-ASN login (login from a new ISP), and unusual-time login (admin login at 3am local time). Anomalies trigger step-up MFA + Slack alert to the security channel. Repeated anomalies on a single account trigger automatic lockout + user notification.

6. JWT Architecture

PreOne uses JSON Web Tokens (JWT) as the stateless authentication credential. JWTs enable horizontal scaling without session affinity, reduce DB load on the auth hot path, and provide a tamper-evident carrier for user identity and authorization context. The trade-off — token revocation is harder — is mitigated by short access-token TTL (15 min), Redis-backed blacklist for revocation, and refresh-token rotation for theft detection.

6.1 Token Issuance Flow

The flow below illustrates the complete token lifecycle from login to refresh. Every step is audited. The rotation pattern (step 7) is critical: each refresh issues a new refresh token and invalidates the prior one. If a stolen refresh token is presented after the legitimate user has already refreshed, the server detects the reuse and revokes the entire session family — a strong signal of token theft.



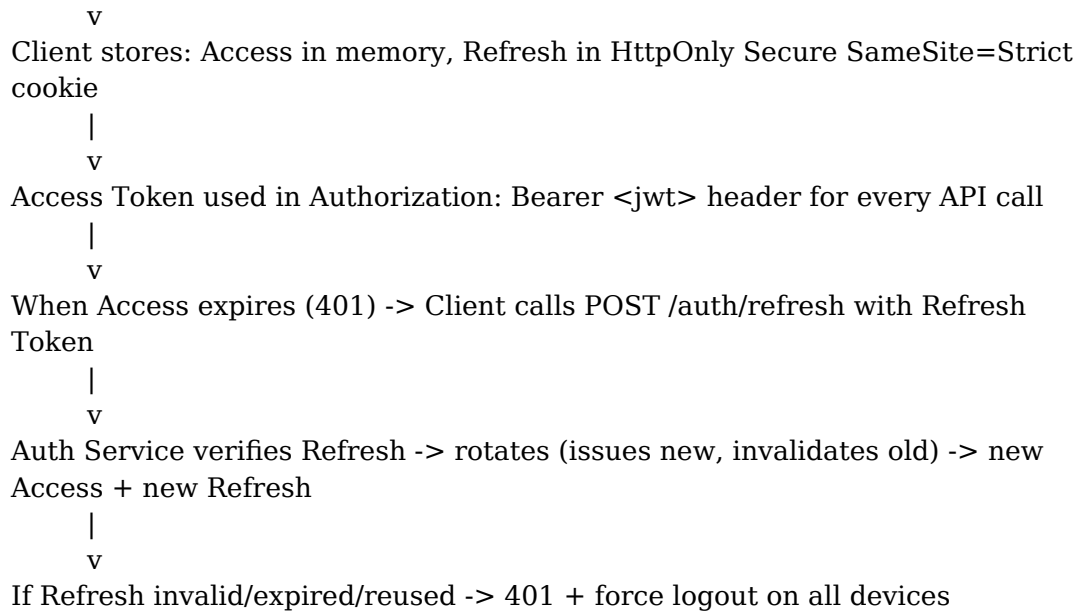


Figure 6.1 — JWT lifecycle: login, access-token use, refresh-token rotation, theft detection

6.2 JWT Claims

The JWT carries the minimum claims necessary to authorize a request without a DB hit. Claims are validated on every request — missing or invalid claims cause 401. The `permissionsVersion` claim is the cache-invalidation mechanism: when a user's roles or permissions change, the server bumps `permissionsVersion`; if the JWT's version is stale, the server rejects the token and forces re-auth, ensuring revocation of stale permissions within 15 minutes.

Claim	Description
<code>sub</code>	Subject — User UUID (primary key in users table)
<code>tenantId</code>	Multi-tenant isolation key. Required on every request. Enforced by Prisma tenant scope.
<code>schoolId</code>	Currently selected school (user may belong to multiple). Drives data scope.
<code>branchId</code>	Currently selected branch. Null for school-wide or multi-branch roles.
<code>roles</code>	Array of role codes (e.g. ["SCHOOL_OWNER", "PRINCIPAL"]). Drives RBAC check.

Claim	Description
permissionsVersion	Integer bumped on every role/permission change for that user. If mismatch with server-side cache, token is rejected and user must re-auth.
sessionId	UUID per login session. Used for session revocation and concurrent-session tracking.
exp	Expiration time (Unix seconds). 15 min for access, 30 days for refresh.
iat	Issued-at time (Unix seconds). Used to detect clock skew and token age.
iss	Issuer — "preone-auth". Verified on every request.
aud	Audience — "preone-api" for access, "preone-auth" for refresh. Prevents token cross-use.

6.3 JWT Security Controls

Beyond the standard JWT validation, PreOne enforces the controls below. These controls address known JWT pitfalls: algorithm confusion attacks (mitigated by RS256 + key-pin), token theft (mitigated by rotation), revocation latency (mitigated by Redis blacklist), and audience confusion (mitigated by iss + aud validation).

Signing Algorithm	RS256 (asymmetric) — private key in AWS KMS, public key cached by API for verification
Key Rotation	Signing key rotated every 90 days; old keys honored for 7-day overlap; kid header identifies key
Token Revocation	Access tokens revoked via Redis blacklist (session_id + exp); refresh tokens revoked via DB delete
Refresh Rotation	Every refresh issues new token + invalidates prior; reuse of prior token triggers revocation of entire session family (theft detection)

Audience Validation	Access tokens rejected on /auth/refresh endpoints; refresh tokens rejected on /api/* endpoints
Issuer Validation	iss claim must equal "preone-auth" — rejects tokens minted by other services
Clock Skew Tolerance	±30 seconds; beyond that, token rejected with 401

6.4 Key Management for JWT Signing

JWT signing uses RS256 (asymmetric). The private key never leaves AWS KMS — the auth service calls KMS to sign, the API services use the cached public key to verify. This separation means a compromised API pod cannot mint tokens. The signing key is rotated every 90 days with a 7-day overlap window during which both old and new keys are honored; the kid header identifies which key signed the token. Key rotation is automated and tested in staging before production.

6.5 Token Revocation

Access tokens are short-lived (15 min) and revocable via a Redis blacklist keyed by sessionId. When a user logs out, an admin force-logs-out a user, or anomaly detection flags a session, the sessionId is added to the blacklist with TTL equal to the remaining access-token lifetime. The auth middleware checks the blacklist on every request. Refresh tokens are revocable via DB delete (refresh tokens are stored hashed in DB for this purpose). On refresh-token reuse detection, the entire session family for that user is revoked.

7. Multi-Factor Authentication (MFA)

MFA adds a second independent factor to authentication, dramatically reducing the risk of credential theft. PreOne mandates MFA for roles with elevated privileges or access to sensitive data (financial, HR, system administration). Optional MFA is offered to all users and encouraged via in-app prompts. The MFA challenge is triggered after primary authentication succeeds but before the JWT is issued.

7.1 Mandatory MFA Roles

The roles below require MFA on every login and step-up MFA for the listed sensitive operations. Step-up MFA requires a fresh OTP (issued within the last 5 minutes) even if the user has a valid JWT; this protects against session hijacking where the attacker has stolen the JWT but not the device.

Role	Description
Super Admin	Platform-level admin. MFA required on every login. Step-up MFA required for tenant deletion, key rotation, role grants.
School Owner	Owner of a school tenant. MFA required for fee configuration, staff dismissal, finance export, and parent data bulk operations.
Principal	MFA required for student record modification, report card finalization, attendance bulk adjustments, parent communication broadcast.
Finance Users	MFA required for invoice generation, fee waivers, refund processing, payroll, and bank reconciliation exports.
HR Users	MFA required for staff onboarding, salary changes, document verification, and resignation processing.

7.2 Supported MFA Factors

PreOne supports multiple factors so users have a fallback if the primary channel is unavailable (e.g., SMS undeliverable). The factor list is below. TOTP via authenticator app is planned for v1.1 and will become the primary factor for super admins because it is offline-capable and not vulnerable to SIM-swap attacks.

Factor	Description
OTP (SMS)	6-digit code via MSG91 SMS gateway. Valid 5 min. Primary MFA factor for all roles. Backup codes provided on enrollment.

Factor	Description
Email OTP	8-digit code to registered email. Used when SMS undeliverable, or as second factor for step-up auth on finance/HR operations.
Authenticator App (Future)	TOTP via Google Authenticator / Authy / 1Password. Planned for v1.1 rollout — will replace SMS as primary factor for super admins.
Backup Codes	10 single-use codes provided at enrollment. Hashed in DB. User prompted to regenerate when 3 or fewer remain.

7.3 MFA Enforcement

MFA enforcement is layered. (1) At login: auth service checks if role requires MFA; if yes, issues an MFA challenge and only mints JWT on successful verification. (2) At step-up: sensitive routes are decorated with `@RequireStepUp`; the middleware checks if the JWT contains a recent `mfaVerifiedAt` claim (within 5 min); if not, returns 403 with a challenge. (3) At session anomaly: if a new device or impossible-travel pattern is detected mid-session, the next request triggers MFA re-challenge. All MFA events are audited.

7.4 MFA Bypass & Recovery

MFA bypass is a high-risk operation and is restricted. If a user loses access to all MFA factors (lost phone + lost backup codes), recovery requires: (1) identity verification by school admin + PreOne support, (2) approval by Super Admin, (3) reset of MFA enrollment, (4) mandatory re-enrollment on next login. The entire bypass flow is audited. Bypass is rate-limited to 3 per tenant per month to detect social-engineering attempts.

8. Authorization

Authorization determines what an authenticated user is allowed to do. PreOne uses Enterprise RBAC (Role-Based Access Control) with multi-tenant scope enforcement and field-level security. Authorization is enforced at four layers — route guard, service layer, ORM row filter, and response serializer — so that bypassing any single layer does not

expose data. This defense-in-depth approach is critical for a multi-tenant SaaS where a single bug could leak one school's data to another.

8.1 Authorization Flow

The flow below illustrates the full authorization lifecycle from JWT issuance to UI rendering. Each step is independently testable and auditable. The `permissionsVersion` cache-invalidation step is the key to revoking stale permissions quickly: when an admin changes a user's roles, the server bumps `permissionsVersion`; on the next request, the JWT's `permissionsVersion` no longer matches the server's, and the user is forced to re-authenticate, picking up the new permissions.

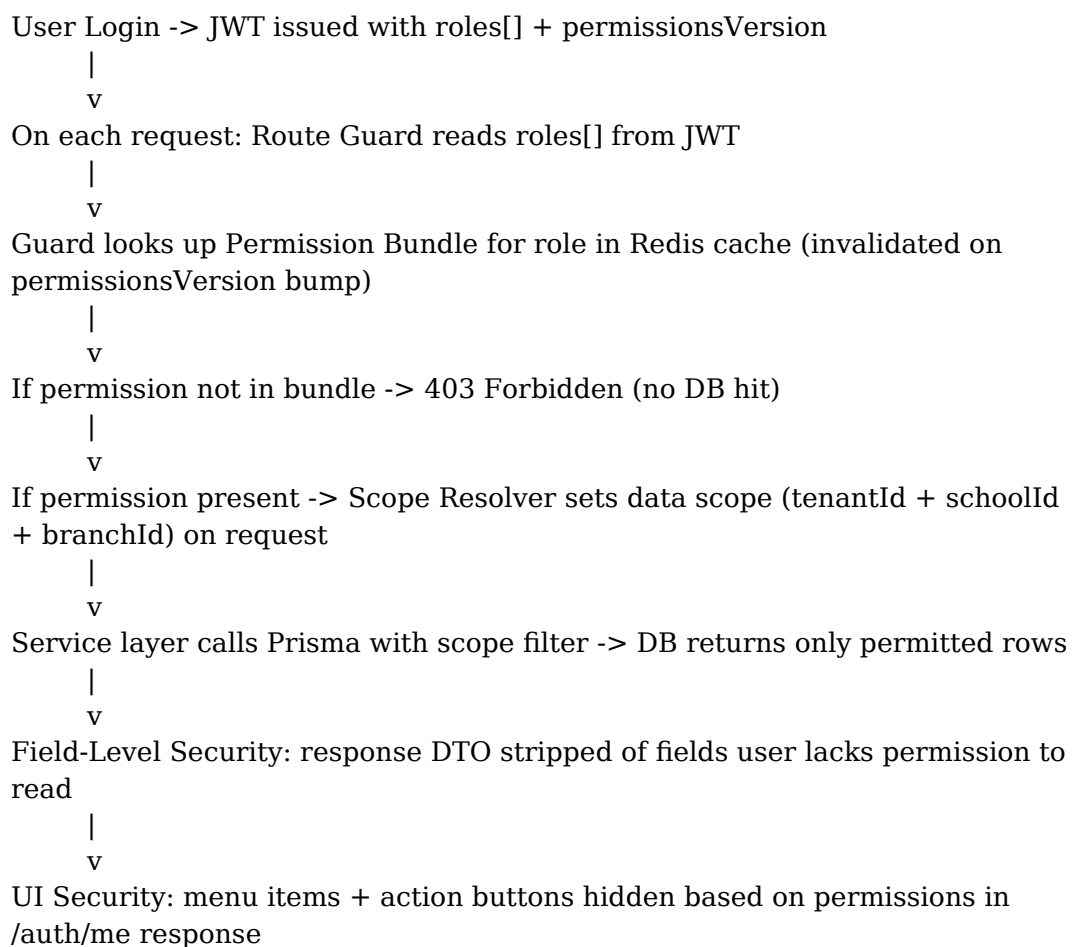


Figure 8.1 — Authorization lifecycle: route guard, permission cache, scope resolution, ORM filter, field security, UI security

8.2 Authorization Features

The features below constitute the PreOne authorization model. Each feature is independently configurable and tested. The combination of role-based + scope-based + field-level security provides fine-grained access control that meets the diverse needs of a

preschool (a teacher sees only her class's students; a principal sees the whole school; a finance user sees fees but not medical records).

Feature	Description
Role-Based Access	User assigned one or more roles (Super Admin, School Owner, Principal, Teacher, Finance, HR, Parent, Student). Each role maps to a bundle of permissions.
Context Switching	Users with multiple roles or multi-school access can switch context (active school, active branch, active role) without re-login. Context stored in JWT.
Data Scope Enforcement	Every query is automatically scoped by tenantId + schoolId + branchId from JWT. A principal of School A cannot see School B's data — enforced at ORM layer.
API Permission Validation	Each route decorated with <code>@RequirePermissions('STUDENT.READ')</code> . Guard checks JWT roles -> permission bundle before controller executes.
Field-Level Security	Response DTOs annotated with <code>@Visibility('FINANCE.SALARY_READ')</code> . Serializer strips fields the requesting user lacks permission for.
Menu & Button Security	Frontend receives permissions list in <code>/auth/me</code> . Menu items, action buttons, and table columns rendered conditionally. Defense in depth — API still enforces.
Approval Workflow	Sensitive actions (fee waiver, salary revision, bulk delete) require secondary approval from a different role. Approval queue + audit trail maintained.
Delegation	Principal can delegate limited permissions to a teacher for a bounded period (e.g., attendance during leave). Delegation auto-expires; audited.

8.3 Permission Bundle Architecture

Permissions are organized into bundles — a bundle is a set of related permissions granted together. For example, the `STUDENT_MANAGEMENT` bundle includes `STUDENT.READ`, `STUDENT.CREATE`, `STUDENT.UPDATE`, `STUDENT.PROMOTE`, `STUDENT.GRADUATE`. Roles are mapped to bundles, not individual permissions — this simplifies role definition and reduces configuration errors. Custom bundles can be created per tenant for school-specific needs (e.g., a custom bundle for a school's unique visitor-check-in workflow). Bundles are versioned in git and reviewed by security.

8.4 Field-Level Security

Some fields are sensitive enough to warrant per-field permissions. For example, `Student.medicalInfo` requires `STUDENT.MEDICAL_READ`; `Staff.salary` requires `FINANCE.SALARY_READ`; `Parent.aadhaar` requires `PARENT.AADHAAR_READ`. Field-level security is implemented via response DTOs annotated with `@Visibility('PERMISSION_CODE')`. The serializer checks the requesting user's permissions and strips fields they lack permission to read. This is defense-in-depth — the underlying query may have returned the field, but the serializer ensures it never reaches the client.

8.5 Approval Workflow

High-impact actions require secondary approval from a different role. Examples: fee waiver requires principal approval; salary revision requires HR + finance approval; bulk student promotion requires principal approval; data export of >10,000 rows requires admin approval. Approval workflow is implemented as a state machine: `PENDING` -> `APPROVED/REJECTED` -> `EXECUTED`. The workflow is auditable end-to-end and visible in the requester's task list. Approvals can be delegated with auto-expiry.

9. Encryption

Encryption protects data confidentiality in transit (over the network) and at rest (in storage). PreOne uses TLS 1.3 for all transit and AES-256 for all rest encryption. For especially sensitive PII, application-level column encryption is layered on top of volume encryption — the application encrypts the field with a per-tenant KMS key before writing to DB, so even a DBA with raw SQL access cannot read the plaintext. This chapter enumerates every encryption control and the data classes that require special handling.

9.1 Encryption in Transit

Every network hop in PreOne is encrypted. The table below enumerates each hop, the TLS version, and the certificate management strategy. The use of a service mesh (Istio/Linkerd) provides zero-config mTLS between microservices — developers do not need to manage service-to-service certificates; the mesh handles issuance, rotation, and verification automatically.

Layer	Technology	Detail
Client ↔ Edge	TLS 1.3	HSTS preload; modern ciphers only (AES-GCM, ChaCha20-Poly1305); no TLS 1.0/1.1/1.2 fallback
Edge ↔ ALB	TLS 1.3	AWS-managed certificate; auto-renewed; ACM integrated
ALB ↔ K8s Ingress	TLS 1.3	Internal CA cert; mTLS optional for service-to-service high-trust paths
Pod ↔ Pod (mTLS)	Istio / Linkerd	Service mesh provides mTLS; zero-config encryption between microservices
App ↔ PostgreSQL	TLS 1.3	rds.force_ssl=1; cert verification; connection pooler (PgBouncer) also TLS
App ↔ Redis	TLS 1.3	Redis 6+ with TLS; AUTH password from Secrets Manager; cert pinning
App ↔ S3/MinIO	TLS 1.3	HTTPS only; bucket policy denies HTTP; SSE-KMS encryption at rest

9.2 Encryption at Rest

At-rest encryption covers all persistent storage: database volumes, Redis persistence, object storage, backups, K8s secrets, and logs. Volume-level encryption protects against physical media theft; column-level encryption protects against DBA / SQL-injection

exposure of PII; K8s etcd encryption protects against etcd snapshot theft. The table below enumerates each scope.

Scope	Technology	Detail
PostgreSQL volumes	AWS EBS encryption (AES-256)	All RDS / EBS volumes encrypted with AWS-managed KMS key. Snapshots inherit encryption.
PostgreSQL columns (PII)	Application-level AES-256-GCM	Aadhaar, parent identity documents, medical data, payment info encrypted via Prisma middleware before insert; key from KMS per tenant.
Redis	AES-256 (in-memory not at rest)	Redis persistence (RDB/AOF) encrypted; keys themselves are non-sensitive (session IDs, cache keys)
Object storage (S3/MinIO)	SSE-KMS (AES-256)	Server-side encryption with customer-managed KMS key; bucket-level default; per-object key optional
Backups	AES-256 + separate KMS key	Backups encrypted with dedicated backup KMS key (separate from runtime); cross-region replicas re-encrypted
K8s Secrets	etcd encryption (AES-256-GCM)	Kubernetes etcd encryption-at-rest enabled; secrets also sealed via SOPS in git
Logs (Loki)	AES-256	Loki volume encrypted; PII never logged (sanitized at app layer before shipping)

9.3 Sensitive Data Classes

The data classes below receive heightened protection beyond standard volume encryption. For each class, the handling column describes the additional controls — typically column-

level encryption, masked display, access logging, and consent requirements. Engineers adding new PII fields must classify them against this table; new classes require security review and an ADR amendment.

Data Class	Handling
Aadhaar (if stored)	Encrypted at column level; masked in UI (only last 4 visible); access logged; never exported in bulk; consent recorded per DPDP.
Parent Identity Documents	Stored in encrypted S3 with signed URL access (5 min TTL); access logged; auto-deleted after verification (per data retention policy).
Student Medical Data	Encrypted at column level; accessible only to nurse + principal + parent; not included in bulk exports; consent flag required.
Payment Information	Card details never stored — tokenized via payment gateway; UPI VPA stored masked; transaction history kept for 7 years per RBI.
API Keys	Stored in AWS Secrets Manager; rotated every 90 days; never logged; access via IAM role only; per-tenant where applicable.
Refresh Tokens	Hashed (SHA-256) before DB storage; single-use; rotation enforced; revocation list in Redis for fast invalidation.

9.4 Key Hierarchy

PreOne uses a three-tier key hierarchy. Tier 1: AWS KMS Customer Master Keys (CMK) — never leave KMS, used only to encrypt/decrypt data-encryption keys. Tier 2: Data Encryption Keys (DEK) — generated per object or per tenant, encrypted by CMK, stored alongside data. Tier 3: Volume-level AWS-managed keys — default EBS/S3 encryption, transparent to the application. This hierarchy means compromising a single DEK exposes only one object/tenant; compromising the CMK is required for large-scale exposure, and CMK access is tightly IAM-scoped + logged.

9.5 Cryptographic Standards

PreOne adheres to the following cryptographic standards: TLS 1.3 (TLS 1.2 deprecated, TLS 1.0/1.1 disabled); AES-256-GCM for symmetric encryption (authenticated encryption — confidentiality + integrity); RSA-2048 / Ed25519 for JWT signing; SHA-256 for hashing (SHA-1/MD5 forbidden); Argon2id for password hashing (BCrypt cost 12 fallback); HMAC-SHA-256 for request signing. These standards are reviewed annually against NIST and IETF recommendations; deprecated algorithms are phased out within 6 months of deprecation.

10. Object Storage Security

Object storage (AWS S3 + MinIO for on-prem) holds PreOne's largest volume of sensitive data — student photos, parent identity documents, medical records, fee receipts, and report cards. Misconfiguration of object storage is the single most common cause of cloud data breaches industry-wide, so PreOne applies a strict controls baseline to every bucket. The principle is simple: no direct public access, ever; all access via short-lived signed URLs issued after authentication, authorization, and audit logging.

10.1 Storage Security Controls

The controls below are applied uniformly to every bucket in every environment. Deviations (e.g., a public bucket for marketing assets) require ADR approval and a named owner. Controls are validated in CI via IaC policy checks — a bucket template missing any control fails the build.

Control	Detail
Server-Side Encryption	SSE-KMS with customer-managed key. Every object encrypted on upload. Bucket policy denies unencrypted uploads.
Signed URLs	All object access via pre-signed URLs with 5-minute TTL (download) or 15-minute TTL (upload). Direct bucket access denied to all IAM principals.
Access Expiry	Signed URL max TTL 15 minutes. Long-term sharing via API endpoint that re-issues signed URL after permission check + audit log.

Control	Detail
Versioning	Bucket versioning enabled. Accidental overwrite/deletion recoverable. MFA delete required for permanent deletion (break-glass only).
Bucket Policy	Public access block at account + bucket level. HTTPS-only enforce. Cross-region replication for backups bucket. Lifecycle policy transitions to Glacier after 90 days.
Access Logging	S3 access logs shipped to Log Archive bucket (immutable, Object Lock enabled). CloudTrail data events captured for all object-level API calls.
Upload Validation	Magic-byte verification + MIME whitelist (jpg, png, pdf, mp4). ClamAV scan on upload. Size cap 10MB (50MB for video). Filename sanitized (UUID only).
Tenant Isolation	Object key prefix includes tenantId. Bucket policy + IAM role scoped to tenant. Cross-tenant access denied at IAM layer; service layer also validates.

10.2 Upload Workflow

File uploads follow a three-step secure workflow. (1) Client requests upload URL via POST /files/upload-url with file metadata (name, size, MIME type). (2) Server validates MIME whitelist, size cap, and tenant quota; issues a pre-signed PUT URL (15 min TTL) targeting a tenant-scoped key (e.g., tenants/{tenantId}/students/{studentId}/documents/{uuid}.pdf). (3) Client uploads directly to S3 via the signed URL — the file never traverses PreOne API servers. After upload, a webhook triggers ClamAV scan + magic-byte verification; if either fails, the object is quarantined and the DB record is marked PENDING_REVIEW.

10.3 Download Workflow

Downloads follow a similar pattern. (1) Client requests download URL via GET /files/{fileId}/download-url. (2) Server validates user permission (field-level), tenant scope,

and audit-logs the request. (3) Server issues pre-signed GET URL (5 min TTL). (4) Client downloads directly from S3. The signed URL is single-use in practice (5 min TTL) and bound to the requesting user via audit log. Bulk downloads (e.g., export of 100 photos) issue a zip-creation job in BullMQ; the user is notified when the zip is ready and receives a single signed URL for the zip.

10.4 Tenant Isolation

Every object key is prefixed with `tenants/{tenantId}/`. IAM roles are scoped per-tenant via bucket policy conditions. Cross-tenant access is denied at the IAM layer — even if the application has a bug that constructs a cross-tenant key, S3 will reject the request because the application's IAM role is not authorized for that tenant prefix. This is the strongest possible isolation: a bug becomes a 403, not a data leak.

10.5 Lifecycle and Retention

Buckets have lifecycle policies: documents transition to Glacier after 90 days (cost optimization); expired documents are auto-deleted per retention policy (7 years for financial, 5 years for academic, immediate on consent withdrawal for marketing). Versioning is enabled on all buckets; MFA delete is required for permanent deletion (break-glass only). Object Lock is enabled on the audit log bucket for WORM (Write Once Read Many) compliance — audit logs cannot be deleted even by an admin.

11. Audit Trail

The audit trail is PreOne's tamper-evident record of every sensitive action. It serves three purposes: forensic investigation (what happened, when, by whom), compliance attestation (DPDP, RBI, internal policies), and anomaly detection (real-time alerts on suspicious patterns). The audit trail is append-only, hash-chained, and retained for 7 years. Audit events are written synchronously in the same transaction as the business operation — if the audit write fails, the business operation rolls back.

11.1 Audited Actions

The actions below are mandatory audit triggers. The list is intentionally broad — when in doubt, audit. Each action captures the listed fields in addition to the standard audit record schema (Section 11.2). Adding a new audited action requires only decorating the route with `@Audit('ACTION_CODE')`; the audit middleware handles capture.

Action	Captured Fields
Login	userId, IP, device, browser, location (city), success/fail, MFA factor used
Logout	userId, sessionId, IP, duration
Failed Login	identifier (phone/email masked), IP, reason (wrong password / wrong OTP / locked)
Create	userId, entity type, entity ID, full snapshot of new record, tenantId, schoolId
Update	userId, entity type, entity ID, old value, new value, changed fields list
Delete	userId, entity type, entity ID, snapshot before deletion, soft-delete flag set
Approvals	approverId, requesterId, entity type, entity ID, action approved, comments
Payments	userId, payerId, amount, gateway, transaction ID, status, invoice ID
Permission Changes	actorId, targetUserId, old roles, new roles, justification
Data Export	userId, export type, filter criteria, row count, file hash, signed URL TTL
File Download	userId, object key, bucket, signed URL issued, IP, UA

11.2 Audit Record Schema

Every audit record conforms to the schema below. The schema is normalized for query efficiency — e.g., `entity_type` + `entity_id` support "show me all actions on Student X" queries; `user_id` supports "show me all actions by User Y" queries; `tenant_id` supports tenant-scoped audit queries. The `hash_chain` field is the tamper-evidence mechanism: each record's hash includes the previous record's hash, forming a chain that cannot be modified without detection.

Field	Type	Description
id	UUID	Primary key, generated server-side
timestamp	TIMESTAMPTZ	Server time at action completion. UTC. Indexed for time-range queries.
user_id	UUID	Actor performing the action. Null for system-generated events.
role_snapshot	JSONB	Roles + permissionsVersion at time of action. Frozen snapshot for forensic accuracy.
ip_address	INET	Source IP from X-Forwarded-For (ALB). Trusted proxy chain validated.
device	TEXT	User-Agent parsed into device + browser + OS. Used for anomaly detection.
action	TEXT	Normalized action code (LOGIN, CREATE, UPDATE, DELETE, APPROVE, PAYMENT, EXPORT, etc.)
entity_type	TEXT	Domain entity (Student, Fee, Attendance, Staff, etc.)
entity_id	UUID	Primary key of affected entity. Null for bulk operations.
old_value	JSONB	Entity snapshot before action. Null for CREATE.
new_value	JSONB	Entity snapshot after action. Null for DELETE.
result	TEXT	SUCCESS / FAILURE / DENIED. Includes error code on failure.

Field	Type	Description
tenant_id	UUID	Tenant scope. Indexed for tenant-scoped audit queries.
request_id	UUID	Correlates with API request ID for end-to-end tracing.
hash_chain	TEXT	SHA-256 of (previous_record_hash + current_record_payload). Tamper-evident chain.

11.3 Hash Chain Integrity

Each audit record's `hash_chain` field is `SHA-256(prev_record_hash + current_record_canonical_json)`. To verify integrity, a background job nightly recomputes the chain from the oldest unverified record forward and compares against stored hashes. Any mismatch triggers a Critical alert — someone has tampered with the audit trail. The tampered record and all subsequent records are quarantined; the security team investigates. This design means an attacker with DB write access cannot selectively edit history without breaking the chain.

11.4 Audit Log Retention & Access

Audit logs are retained for 7 years (regulatory + internal policy). Hot retention (90 days) is in PostgreSQL for fast querying. Cold retention (7 years) is in S3 with Object Lock (WORM). Audit access is restricted: (1) the security team has read access via a dedicated audit-reader role; (2) tenant admins can read their own tenant's audit via the `/audit` API (rate-limited, paginated, export-logged); (3) super admins can read all audits but every read is itself audited. Bulk audit export requires CISO approval.

11.5 Audit Query API

The `/audit` API exposes audit data to authorized users. Supported queries: by user, by entity, by action, by time range, by tenant. Results are paginated (max 100 per page) and rate-limited (10 requests/min). Each query is itself audited (meta-audit). Export to CSV is supported for compliance reporting; exports are signed and stored in S3 with a 24-hour signed URL. The API enforces tenant scope — a tenant admin cannot query another tenant's audit.

12. DPDP Compliance (India)

The Digital Personal Data Protection Act, 2023 (DPDP) is India's comprehensive data protection law. PreOne, processing personal data of children (a special category under DPDP), must adhere to elevated standards: verifiable parental consent, purpose limitation, data minimization, and the rights to access, correct, and erasure. This chapter maps DPDP requirements to PreOne's architectural controls. Compliance is reviewed annually by external counsel and attested in customer-facing security documentation.

DPDP compliance is not a checkbox exercise — it is a design constraint that influences product decisions. For example, the consent capture screen at enrollment is not a marketing afterthought but a regulated gate that blocks data collection until explicit, granular consent is recorded. Similarly, the right-to-erasure workflow is not a support ticket but an automated pipeline with legal-hold checks, certificate-of-destruction generation, and audit trail.

12.1 Key DPDP Controls

The controls below are PreOne's implementation of DPDP requirements. Each control has a named owner, an automated test, and a metric tracked on the compliance dashboard.

Control	Implementation Detail
Explicit Parent Consent	Consent captured at student enrollment with timestamp, IP, device, and granular purpose flags (academic, medical, photos, marketing). Withdrawable at any time.
Purpose Limitation	Data collected for one purpose (e.g., medical emergency) cannot be repurposed (e.g., marketing) without fresh consent. Purpose tag on every PII field.
Data Minimization	Forms collect only fields needed for stated purpose. Optional fields clearly marked. Annual review of field usage; unused fields purged.
Consent Withdrawal	Parent can withdraw consent via app or support request. Withdrawal triggers data quarantine within 7 days, deletion within 30 days (except where legal retention applies).

Control	Implementation Detail
Data Retention Policy	Student records retained 5 years post-departure (academic requirement). Medical data 7 years (RBI/insurance). Financial 7 years (Income Tax). Then auto-purged.
Right to Correction	Parent can request correction via in-app form. SLA: 7 days. Correction logged in audit trail. Original value preserved in audit history.
Right to Erasure	Parent can request erasure (subject to legal retention). Process: verify identity -> quarantine -> legal hold check -> purge -> certificate of destruction.
Audit Logging	Every PII access, modification, and disclosure logged with actor, purpose, and timestamp. Logs immutable. Available to data principal on request.
Data Residency (India)	All production data in AWS ap-south-1 (Mumbai). Backups in ap-south-1b. No cross-border transfer. DPDP-compliant by design.
Breach Notification Workflow	Breach detected -> contain -> assess scope -> notify DPO within 1 hour -> notify Data Protection Board within 72 hours -> notify affected users with remediation steps.

12.2 Consent Capture

Consent is captured at student enrollment via a multi-step screen that presents each data category (academic, medical, photos, marketing, third-party sharing) separately. The parent must affirmatively opt-in to each category; no pre-checked boxes. The consent record stores: parent identity, timestamp, IP, device, granular flags, consent text version, and a digital signature. Consent can be withdrawn at any time via the parent app or via support request; withdrawal triggers the data quarantine workflow.

12.3 Children's Data

DPDP imposes additional obligations for children's data (under 18). PreOne requires verifiable parental consent before processing any child's data. Tracking of children's data is

restricted — PreOne does not track behavioral data for targeted advertising (and does not engage in behavioral advertising at all). Targeted advertising directed at children is prohibited. These restrictions are encoded in the consent flow and in the data-classification policy.

12.4 Data Principal Rights

Data principals (parents, on behalf of students; staff, on behalf of themselves) have the following rights, accessible via in-app workflows: (1) Right to Access — request a copy of all data PreOne holds about them; (2) Right to Correction — request correction of inaccurate data; (3) Right to Erasure — request deletion (subject to legal retention); (4) Right to Grievance Redressal — contact the Grievance Officer whose details are published in the privacy policy. SLAs: Access 7 days, Correction 7 days, Erasure 30 days, Grievance 30 days.

12.5 Breach Notification

If a personal data breach occurs, PreOne follows the breach notification workflow: (1) Detection (automated monitoring or manual report); (2) Containment (within 1 hour — isolate affected systems, rotate credentials); (3) Assessment (within 4 hours — scope, severity, affected principals); (4) Internal notification (DPO within 1 hour, leadership within 4 hours); (5) Regulator notification (Data Protection Board within 72 hours, per DPDP Section 8(6)); (6) Principal notification (affected users without undue delay, with remediation guidance). The workflow is tested via tabletop exercises quarterly.

13. Key Management

Cryptographic keys are the foundation of every other security control — if keys are compromised, encryption, signing, and authentication all fall. PreOne centralizes key management in AWS KMS + Secrets Manager, with strict access controls, automated rotation, and complete audit logging. No human ever reads a production secret in plaintext; secrets are mounted into pods at runtime via the External Secrets Operator and never written to disk.

13.1 Secret Stores

PreOne uses four distinct stores for secrets, each with a specific purpose. The separation ensures that compromise of one store does not expose all secrets. For example, git-sealed secrets (SOPS) are version-controlled and reviewable but contain only non-sensitive config; runtime secrets (K8s Secrets) are memory-backed and ephemeral; production secrets

(Secrets Manager) are the source of truth with rotation; encryption keys (KMS) are non-extractable and used only via API.

Store	Description
AWS Secrets Manager	Primary store for all secrets at rest. Auto-rotation for DB credentials, Redis AUTH, JWT signing keys. Encrypted with KMS. IAM-scoped access per service role.
Kubernetes Secrets	Runtime-mounted secrets for pods. Synced from Secrets Manager via External Secrets Operator. Memory-backed (tmpfs) — never written to disk.
AWS KMS	Customer-managed keys (CMK) for envelope encryption. Per-tenant CMK for column-level PII. Per-environment CMK for volume encryption. Key rotation annual.
SOPS + Git (Sealed)	Non-sensitive config (timeout values, feature flags) versioned in git via SOPS (encrypted with KMS). Reviewable in PR. No secrets in plaintext in git.

13.2 Managed Keys

The keys below are managed by PreOne with the listed rotation cadence. Rotation is automated where possible (Secrets Manager integration); manual rotation requires a runbook + dual-control (two engineers). All rotations are audited and tested in staging before production. The rotation-overlap window (e.g., 7 days for JWT) ensures in-flight tokens remain valid during rotation.

Key	Rotation	Detail
JWT Signing Keys	90 days	RS256 keypair. Private key in KMS. Public key cached by API. kid header identifies version. 7-day overlap during rotation.

Key	Rotation	Detail
Database Credentials	30 days	RDS master + app user. Rotated via Secrets Manager + RDS integration. App picks up new creds via connection pooler reload.
Payment Gateway Keys	Per gateway policy	Razorpay, Cashfree, Stripe API keys. Rotated per vendor guidance. Stored in Secrets Manager. Used only by payment service.
WhatsApp API Keys	60 days	Meta WhatsApp Business API token + phone number ID. Rotated via Meta Business Manager. Secret rotation triggers config reload.
Email Credentials	90 days	SES SMTP credentials. Per-environment. Used by notification service. IAM-scoped to notification service role only.
Firebase Keys	Per Google rotation	Server key + service account JSON for FCM. Used by notification service. Stored in Secrets Manager. Access scoped to notification namespace.
Encryption Keys (column-level)	Annual	Per-tenant KMS CMK for PII column encryption. Old keys retained for decrypt of historical data. Re-encryption job runs after rotation.
Redis AUTH	30 days	Per-environment Redis password. Rotated via Secrets Manager. App reloads connection pool on rotation signal.

13.3 Key Rotation Process

Key rotation follows a four-step process. (1) New key is generated in KMS/Secrets Manager. (2) Application is configured to honor both old and new keys during overlap window (e.g., JWT verifier accepts both kids; DB connection pool supports credential reload). (3) Old key is marked inactive but not deleted — used only for decrypt/verify of historical data. (4) After overlap window, old key is revoked (delete scheduled per retention policy). The process is fully automated for Secrets Manager-managed keys; manual keys follow the same process via runbook.

13.4 Emergency Key Revocation

In the event of suspected key compromise, emergency revocation is available 24/7. The process: (1) On-call engineer triggers revocation via break-glass tooling (requires dual approval); (2) KMS key is disabled immediately — all encrypt/decrypt operations fail; (3) Affected services degrade gracefully (fail secure); (4) New key is generated and deployed; (5) Re-encryption job runs for data encrypted with the compromised key. The process is tested quarterly via game-day exercises. Target time-to-revoke: 15 minutes from trigger.

13.5 Secret Access Audit

Every secret access is logged. Secrets Manager access is logged via CloudTrail; KMS key usage is logged via KMS API events; K8s Secret access is logged via audit-policy. Logs are aggregated in SIEM and correlated with application logs — e.g., a Secrets Manager access without a corresponding deployment is flagged as anomalous. Quarterly access review: each secret's access list is reviewed by the secret owner; access no longer needed is revoked. Annual attestation: each team confirms their secret inventory.

14. API Security

The API is PreOne's primary attack surface — every protected resource is exposed via an API endpoint. API security is therefore the most heavily controlled layer. This chapter enumerates the controls applied to every `/api/*` route. Controls are enforced in middleware so they cannot be forgotten by a developer adding a new route — the framework applies them by default, and a developer must explicitly opt out (with security review) to skip any control.

14.1 API Security Controls

The controls below are applied uniformly to every API route. Each control has a corresponding automated test in the test suite — a route missing a control fails the build. Controls are layered; a request must pass all controls to reach the controller logic.

Control	Detail
JWT Authentication	Every /api/* route requires Bearer token except /auth/login, /auth/refresh (which uses refresh cookie), and /health. 401 on missing/invalid token.
Rate Limiting	Per-IP: 1000 req/min. Per-token: 600 req/min default, 100 req/min on sensitive endpoints (auth, payment, export). 429 with Retry-After header on exceed.
Request Validation	class-validator DTOs on every controller. Reject 400 on schema mismatch. Whitelist enabled — unknown fields rejected. Forbid non-whitelisted properties.
Response Sanitization	Response DTOs use class-transformer @Exclude / @Expose. Sensitive fields stripped based on requesting user's permissions. No raw Prisma objects leaked.
Idempotency Keys	POST /payment, POST /fees, POST /admission require Idempotency-Key header. Server caches response for 24h. Duplicate key with different payload returns 409.
API Versioning	URI versioning (/v1, /v2). Deprecation header (Sunset) on old versions. 12-month deprecation window. Breaking changes only in major version.
Secure Headers	Helmet middleware sets HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy. CSP with nonce. No X-Powered-By.
CORS Policy	Allowlist of origins (app.preone.in, admin.preone.in, parent.preone.in). Credentials allowed. Preflight cached 24h. No wildcard origins.

Control	Detail
Request Size Limit	Body max 1MB for JSON, 10MB for multipart (file upload). 413 on exceed. Multipart parsing via busboy with field count + size limits.
SQL Injection Prevention	Prisma ORM with parameterized queries only. No raw SQL. class-validator on inputs. WAF rule for SQLi patterns. Periodic SAST scan.
XSS Prevention	React escapes by default. CSP header. DOMPurify on rich text (tinymce output). No dangerouslySetInnerHTML. Output encoding on API responses.

14.2 Rate Limiting Strategy

Rate limiting is the primary defense against API abuse. PreOne uses a tiered strategy: (1) Per-IP limit (1000 req/min) — blocks DDoS and credential stuffing from a single IP; (2) Per-token limit (600 req/min default, 100 req/min on sensitive endpoints) — blocks token abuse; (3) Per-tenant aggregate limit (10000 req/min) — protects shared infrastructure from a single noisy tenant. Limits are enforced in NGINX (LUA + Redis) for performance. On limit exceed, 429 with Retry-After header; persistent abuse triggers IP ban via WAF.

14.3 Idempotency

State-changing operations (POST /payment, POST /fees, POST /admission) require an Idempotency-Key header. The server caches the response (keyed by idempotency-key + tenant + user) for 24 hours. A duplicate request (same key) returns the cached response; a request with the same key but different payload returns 409 Conflict. This protects against network-retry-induced double-charges and is critical for payment reliability. The idempotency cache is in Redis with TTL 24h.

14.4 API Versioning

API versioning is URI-based: /v1, /v2. Breaking changes require a new major version; non-breaking changes (additive) ship in the existing version. Old versions are supported for 12 months after deprecation, with a Sunset response header indicating the deprecation date.

Customers are notified 90 days before deprecation. The current production version is v1; v2 is planned for Q1 2027 with breaking changes to the fee and attendance APIs (unified schema).

14.5 GraphQL (Future)

If GraphQL is introduced (under consideration for v2), additional controls will apply: query depth limiting (max 7), query complexity scoring (max 1000 points), persisted queries only (no arbitrary client queries), per-field authorization, and rate limiting per query complexity. GraphQL endpoints will be reviewed by security before launch.

15. Security Monitoring

Security monitoring converts raw logs into actionable alerts. PreOne's monitoring pipeline aggregates signals from WAF, ALB, application audit logs, K8s audit logs, CloudTrail, and database query logs into a SIEM (Security Information and Event Management) system. The SIEM applies correlation rules to detect attack patterns that no single signal would reveal. Alerts are routed by severity to PagerDuty (High), Slack (Medium), or daily digest (Low).

15.1 Monitored Signals

The signals below are continuously monitored. Each signal has a threshold (detection criterion) and an automated action. Thresholds are tuned quarterly based on false-positive rates and changing attack patterns. The SIEM correlates signals — e.g., a failed login followed by a successful login from a new device within 5 minutes triggers a higher-severity alert than either signal alone.

Signal	Threshold	Action
Failed Logins	>10 per IP in 5 min OR >5 per account in 15 min	IP rate limit + account lockout + Slack alert to security channel
Privilege Escalation Attempts	Any 403 on /admin/* or /permissions/* endpoint	Immediate PagerDuty alert + user flagged for review
Suspicious API Calls	Unusual endpoint pattern (e.g., student endpoint hit by finance role) >50 in 1h	Auto-throttle + alert + SIEM correlation

Signal	Threshold	Action
Token Abuse	Refresh token reuse detected (same token presented twice)	Session family revoked + user notified + investigation
SQL Injection Attempts	WAF rule match for SQLi pattern	Request blocked + IP flagged + 3 strikes -> IP ban
DDoS Events	Requests/sec > 5x baseline for 2 min	AWS Shield engaged + CloudFront challenge + auto-scale
Malware Uploads	ClamAV positive on file upload	File quarantined + upload rejected + user flagged + admin notified
Bulk Data Export	Export >10,000 rows in single request OR >3 exports by same user in 1h	Approval required + DLP scan + audit alert
After-Hours Admin Activity	Admin action outside 7am-9pm IST on weekends/holidays	MFA re-challenge + Slack alert to security channel
New Device Login	First login from unrecognized device fingerprint	OTP challenge + email notification to user

15.2 Alert Severities

Alerts are classified into three severities with corresponding response SLAs. The classification is automated by playbook rules but can be overridden by the on-call engineer. Severities are reviewed monthly — a High that consistently turns out to be a false positive is demoted to Medium; a Low that consistently indicates real issues is promoted.

Severity	Response SLA	Examples
High	Immediate (PagerDuty + call)	Active breach, data exfiltration, ransomware, auth service compromise, key leakage, DB exposure detected

Severity	Response SLA	Examples
Medium	Within 15 minutes (Slack + on-call)	Suspicious API pattern, repeated failed logins, config drift, expired cert detected, WAF rule surge
Low	Daily review (Slack digest)	Single failed login anomaly, low-rate bot traffic, dependency CVE (low severity), informational audit events

15.3 SIEM Correlation Rules

Beyond single-signal alerts, the SIEM applies correlation rules that combine multiple signals over time windows. Examples: (1) Failed login from IP X followed by successful login from same IP within 1 hour → credential stuffing suspected; (2) User access to /audit/export followed by large S3 download within 30 min → data exfiltration suspected; (3) Config change (Terraform apply) outside business hours → investigate; (4) Multiple 403s on /admin endpoints from same user within 5 min → privilege escalation attempt. Correlation rules are versioned in git and reviewed quarterly.

15.4 Security Dashboard

A real-time security dashboard (Grafana) is displayed in the engineering office and accessible to all engineers. It shows: active incidents, open alerts, failed-login rate, WAF block rate, dependency CVE count, certificate expiry timeline, and audit log integrity status. The dashboard serves both operational awareness and cultural reinforcement — security is visible, not hidden behind a tool.

15.5 Threat Intelligence

PreOne subscribes to threat intelligence feeds (AlienVault OTX, AbuseIPDB) that feed the WAF and SIEM. Known-malicious IPs are blocked at WAF; known-malicious user-agents are flagged in SIEM. The feeds are updated hourly. Additionally, PreOne participates in ISAC (Information Sharing and Analysis Center) for the education sector, sharing anonymized indicators of compromise and receiving indicators from peers.

16. Incident Response

Despite preventive controls, security incidents will occur. PreOne maintains a structured incident response process to minimize impact, restore service, and learn from the incident. The process is documented, tested via tabletop exercises quarterly, and improved after every real incident via blameless postmortem. The goal is not zero incidents (unrealistic) but rapid detection, fast containment, and continuous improvement.

16.1 Incident Response Phases

The six phases below structure the response to any security incident. Each phase has entry criteria, exit criteria, and a named owner. The process is sequential but with feedback loops — e.g., investigation may uncover new containment actions. The Incident Commander owns the overall process and decides phase transitions.

Phase	Detail
1. Threat Detected	Signal from monitoring (WAF, SIEM, anomaly detection, user report, third-party notification). Auto-correlated in SIEM. Severity assigned by playbook rules.
2. Alert Generated	PagerDuty page for High; Slack alert for Medium; daily digest for Low. On-call engineer acknowledges within SLA. Incident channel spun up automatically.
3. Auto Containment	Where playbook permits, automated containment kicks in: IP ban, account lock, pod quarantine, secret rotation trigger, WAF rule update, traffic shaping.
4. Security Investigation	On-call + security engineer pull logs, audit trail, network captures. Determine scope: which tenants, which records, which systems. Time-boxed to 60 min for High.
5. Recovery	Patch / rotate / restore. Validate integrity via audit trail hash chain. Re-deploy from known-good image. Re-issue tokens for affected sessions. Monitor for re-occurrence.

Phase	Detail
6. Post-Incident Review	Within 5 business days: blameless postmortem, root cause, timeline, impact, what-went-well, what-went-wrong, action items with owners + due dates. Published to engineering wiki.

16.2 Incident Roles

Clear roles prevent confusion during an incident. The roles below are pre-assigned and rotated weekly (on-call). For High-severity incidents, all roles are activated; for Medium, on-call + security engineer; for Low, on-call handles alone. Roles are documented in the on-call runbook with phone numbers and escalation paths.

Role	Description
On-Call Engineer	First responder. Acknowledges alert, performs initial triage, executes playbook. Time-boxes investigation. Escalates if needed.
Security Engineer	Joins for Medium/High. Performs forensic analysis, coordinates containment, liaises with platform team. Owns the security incident record.
Incident Commander	Assigned for High. Coordinates cross-team response, makes go/no-go decisions on shutdown, communicates with leadership. Single source of truth.
DPO (Data Protection Officer)	Engaged if PII involved. Drives DPDP breach notification workflow. Liaises with Data Protection Board if reportable.
Communications Lead	Drafts internal + external communications. Coordinates with leadership on customer notification timing and tone.

16.3 Incident Severity Classification

Severity is assigned at detection and may be revised as the incident progresses. Critical: active data exfiltration, auth service compromise, ransomware, child data exposure. High: confirmed unauthorized access, large-scale PII exposure, payment system compromise. Medium: suspected unauthorized access, isolated PII exposure, recurring WAF bypass. Low: single anomalous event, suspected but unconfirmed probe. Severity drives response SLA, role activation, and communication cadence.

16.4 Post-Incident Review

Within 5 business days of incident closure, a blameless postmortem is published. The format: (1) Timeline — minute-by-minute from detection to closure; (2) Impact — users/data/systems affected; (3) Root Cause — technical + process; (4) What Went Well — actions that limited impact; (5) What Went Wrong — failures in detection, containment, communication; (6) Action Items — owners + due dates, tracked to closure. The postmortem is shared company-wide; action items are tracked in the security backlog with quarterly review.

16.5 Incident Communication

Communication during an incident follows a tiered approach. Internal: incident channel (Slack) for real-time coordination; incident bridge (Zoom) for voice; status page for engineering-wide visibility. External: customer communication via status page + email for High/Critical; regulator communication per DPDP (Section 12.5); press communication via Communications Lead only. The principle is: be transparent, be timely, but do not speculate — share what is known, what is being done, and when the next update will come.

17. Security Testing

Security testing validates that controls work as designed and surfaces vulnerabilities before attackers do. PreOne runs a layered testing program: automated tests in CI (SAST, dependency scan, secret scan, image scan) catch the common cases on every PR; periodic deeper tests (DAST, API security, pentest) catch issues automated tools miss. Testing is not optional — a PR that fails a security test cannot merge, and a release that fails a security gate cannot ship.

17.1 Security Test Suite

The tests below are the PreOne security test suite. Each test has a tool, a cadence, and a defined remediation SLA based on finding severity. Tests are integrated into CI where possible (SAST, dependency, secret, image) and run as scheduled jobs for the rest (DAST, pentest).

Test	Tool	Cadence	Description
SAST (Static Analysis)	SonarQube + Semgrep	Every PR + nightly main	Scans source for injection, hardcoded secrets, weak crypto, insecure patterns. Fails PR on High.
DAST (Dynamic Analysis)	OWASP ZAP	Nightly on staging	Black-box scan of running app. Crawls routes, fuzzes inputs, detects XSS/SQLi/auth issues. Findings triaged within 48h.
Dependency Scanning	Snyk + npm audit + Dependabot	Every PR + daily	CVE check on all dependencies. Auto-PR for patch updates. Block merge on Critical/High with available fix.
Secret Scanning	git-secrets + TruffleHog + GitHub Secret Scan	Pre-commit + PR + daily	Detects AWS keys, DB credentials, tokens in git history + working tree. Blocks commit on hit. Auto-rotates if leaked.
Container Image Scanning	Trivy + ECR scan-on-push	Every build	Scans image for OS package CVEs + app dependency CVEs + misconfig. Blocks deploy on Critical with fix available.

Test	Tool	Cadence	Description
Penetration Testing	External vendor + internal	Quarterly + pre-major-release	Manual + automated pentest by certified team. Covers OWASP Top 10, business logic, multi-tenant isolation. Findings fixed + re-tested.
API Security Testing	Burp Suite + custom scripts	Monthly + pre-release	Fuzzes API endpoints, tests auth/authz boundary, BOLA/IDOR, rate limit bypass, JWT tampering, mass assignment.
OWASP ZAP Testing	OWASP ZAP baseline + full scan	Nightly baseline + weekly full	Automated web app scan. Baseline runs in CI; full scan in staging with auth context. Findings feed into backlog.

17.2 Remediation SLAs

Findings from security tests have severity-based remediation SLAs. Critical: 24 hours (block release if unfixed). High: 7 days. Medium: 30 days. Low: 90 days. SLAs are tracked on the security dashboard; overdue findings are escalated to engineering leadership. False positives are suppressed with written justification + re-validation date. SLA breaches are reviewed monthly by the security steering committee.

17.3 Penetration Testing Cadence

External penetration testing is conducted quarterly by a certified third party and before every major release (defined as a version bump with breaking schema changes). The scope covers all OWASP Top 10 risks, business-logic attacks (e.g., can a parent see another parent's child?), multi-tenant isolation, and role escalation. The pentest report is reviewed

by engineering + security; findings are tracked to closure with re-test by the pentest vendor. Annual summary is shared with enterprise customers under NDA.

17.4 Bug Bounty (Future)

PreOne plans to launch a private bug bounty program in v1.1, scoped to the staging environment initially and expanding to production in v1.2. The program will reward responsible disclosure of security vulnerabilities with bounties tiered by severity. The program will be managed via HackerOne or Bugcrowd. The disclosure policy will follow ISO 29147 (timely patches, coordinated disclosure).

18. Security Headers

HTTP security headers instruct the browser to apply client-side protections: enforce HTTPS, restrict resource origins, prevent clickjacking, disable MIME sniffing. PreOne sets the headers below on every response via the Helmet middleware in NestJS. Headers are validated in CI — a missing header on any route fails the security-header check. The CSP is the most complex header; it is generated per-request with a fresh nonce for script-src.

18.1 Security Headers Reference

The headers below are set on every HTTP response. The Value column shows the production configuration; staging may have a slightly looser CSP for debugging. The Description column explains the protection each header provides.

Header	Value	Description
Strict-Transport-Security	max-age=63072000; includeSubDomains; preload	Forces HTTPS for 2 years, including subdomains. Preloaded in browser HSTS list.
Content-Security-Policy	default-src 'self'; script-src 'self' 'nonce-{nonce}'; style- src 'self' 'unsafe-inline'; img- src 'self' data: https;; connect-src 'self' https://api.preone.in; frame- ancestors 'none'; base-uri 'self'; form-action 'self'	Restricts resource loading. Nonce-based script CSP. No framing allowed (clickjacking protection).

Header	Value	Description
X-Frame-Options	DENY	Defense-in-depth against clickjacking. CSP frame-ancestors is primary; this is fallback for old browsers.
X-Content-Type-Options	nosniff	Prevents MIME-type sniffing. Browser honors declared Content-Type.
Referrer-Policy	strict-origin-when-cross-origin	Sends origin (not full URL) for cross-origin requests. No referrer for downgrade HTTPS->HTTP.
Permissions-Policy	camera=(), microphone=(), geolocation=(self), payment=(self)	Disables camera/mic entirely. Geolocation + payment allowed only on same origin.
Cross-Origin-Opener-Policy	same-origin	Isolates browsing context. Prevents Spectre-class attacks via cross-origin window references.
Cross-Origin-Resource-Policy	same-site	Restricts resource loading to same-site. Defense against cross-origin resource fetch attacks.

18.2 Content-Security-Policy Deep Dive

CSP is the primary XSS mitigation. PreOne's CSP uses nonce-based script-src: every <script> tag includes a nonce attribute that matches a per-request-generated value; scripts without the matching nonce are blocked. This blocks injected scripts (e.g., from XSS or supply-chain compromise) because they cannot know the nonce. Style-src allows 'unsafe-inline' because CSS injection is lower-risk and many libraries (e.g., styled-components) require inline styles. Frame-ancestors 'none' blocks all framing — strong clickjacking protection. The CSP is tested via the CSP Evaluator tool and reviewed quarterly.

18.3 Header Validation

A CI job runs Mozilla Observatory and securityheaders.com against staging after every deploy. Score below A+ fails the build. The job also validates that headers are present on error pages (401, 403, 404, 500) — a common oversight that leaves error pages unprotected. Header drift (a route missing a header) is detected via a scheduled canary that hits every public route and checks header presence.

19. Final Security Stack

The table below is the consolidated PreOne security stack — every layer, the chosen technology, and its purpose. This is the authoritative reference for security architecture decisions; changes to any entry require an ADR amendment and security team approval. The stack is reviewed annually against industry benchmarks (NIST CSF, CIS, OWASP) and updated as threats evolve.

Layer	Technology	Purpose
Authentication	JWT (RS256, 15 min) + Refresh Token (30 d, rotated) + OTP (SMS/Email)	Stateless auth, short-lived tokens, refresh rotation for theft detection
MFA	SMS OTP + Email OTP + Backup Codes (TOTP planned v1.1)	Step-up auth for privileged roles; theft-resistant via rotation
Authorization	Enterprise RBAC + Scope (tenant/school/branch) + Field-Level Security	Fine-grained, multi-tenant access control with defense-in-depth
Password Hashing	Argon2id (preferred) / BCrypt cost 12	Memory-hard hashing resistant to GPU/ASIC brute force
Encryption (Transit)	TLS 1.3 everywhere + mTLS service mesh (Istio)	Confidentiality + integrity on every hop
Encryption (Rest)	AES-256-GCM (EBS, S3, column-level PII)	Data unreadable if storage media compromised
Secrets	AWS Secrets Manager + KMS + SOPS + K8s External Secrets	Centralized, rotated, IAM-scoped secret management

Layer	Technology	Purpose
Audit	Immutable audit trail (PostgreSQL + hash chain) + Loki logs	Tamper-evident record of every sensitive action
API Security	Rate Limiting + Validation + Idempotency + Secure Headers	Protect API surface from abuse + injection + replay
Compliance	DPDP-aligned controls + consent management + data residency (India)	Regulatory compliance for Indian personal data law
Monitoring	SIEM + Prometheus + Grafana + Loki + AlertManager	Real-time detection of anomalies + attacks
Security Testing	SAST + DAST + Dependency Scan + Secret Scan + Image Scan + PenTest	Continuous validation of security posture

19.1 Stack Validation

Every layer in the stack has an owner, a metric, and a validation cadence. The owner is accountable for the layer's effectiveness. The metric is tracked on the security dashboard (e.g., for Authorization, the metric is unauthorized-access attempt rate from DAST). The validation cadence is the frequency of automated + manual testing (e.g., for Encryption, validation is annual crypto-agility drill + quarterly key-rotation test). Layers without a clear owner or metric are flagged for remediation in the quarterly security review.

19.2 Stack Evolution

The security stack is not static. Planned evolution over the next 12 months: (1) TOTP via authenticator app as primary MFA factor for super admins (Q3 2026); (2) Hardware security key (WebAuthn / FIDO2) support for super admins (Q1 2027); (3) Service mesh with mTLS across all namespaces (Q4 2026); (4) Private bug bounty program launch (Q4 2026); (5) SOC 2 Type II attestation (Q2 2027); (6) ISO 27001 certification (Q4 2027). Each item has a named owner and is tracked in the security roadmap.

20. Cross References

This ADR does not stand alone — it is part of the PreOne architecture baseline. The references below link this ADR to predecessor and successor documents. Engineers implementing security controls should consult these references for full context. In particular, ADR-006 through ADR-026 define the role taxonomy that this ADR enforces, and the API Contract Catalog defines the concrete API contracts for /auth, /permissions, and /audit endpoints.

Reference	Topic	Description
ADR-006 – ADR-026	Identity, Roles, Authorization, Multi-Tenancy & System Roles	Defines the role taxonomy, permission bundles, scope model, and system roles referenced throughout this ADR.
BRC v1.0	Compliance, Data Governance, Notification, Approval and Audit Rules	Business rules layer: consent capture, approval workflow, audit triggers, retention SLAs referenced by DPDP controls.
API Contract Catalog v1.0	Authentication, Authorization, JWT, Security Headers and Rate Limiting	Concrete API contracts for /auth/*, /permissions/*, /audit/*, plus security header + rate limit specifications.
ERD v3.0	Audit tables, multi-tenant schema and security-related entities	Physical schema for audit_log, refresh_token, consent, device_fingerprint, secret_version tables.
ADR-111 DevOps & Infrastructure v1.0	K8s namespaces, network policies, backup strategy, DR	Infrastructure-level controls: network isolation, pod security, backup encryption, DR failover referenced by incident response.
OpenAPI Specification v1.0	Security schemes, scopes, error responses	Bearer JWT + scope definitions on every endpoint, 401/403/429 response contracts.

20.1 Document Control

Document ID	ADR-113
Title	Enterprise Security Architecture
Version	1.0
Status	LOCKED
Date	2026-07-14
Owner	PreOne Architecture & Engineering Team
Review Cadence	Quarterly + after major incidents
Next Review	2026-10-14
Change Control	Amendment requires Architecture Review Board approval
Distribution	Internal Engineering — Confidential

21. Glossary

The glossary below defines security terms used throughout this ADR. Terms are listed alphabetically. Where a term has multiple meanings in industry usage, the PreOne-specific meaning is given.

Term	Definition
Zero Trust	Security model that assumes no implicit trust — every request is verified regardless of source network.
RBAC	Role-Based Access Control — permissions assigned to roles, users inherit permissions via role assignment.
Scope	Data boundary (tenant + school + branch) that limits which records a user can access, enforced at ORM layer.
JWT	JSON Web Token — compact, signed token carrying claims (sub, roles, exp) used for stateless authentication.

Term	Definition
Refresh Token Rotation	Pattern where each refresh issues a new token + invalidates the prior; reuse of prior token triggers session revocation (theft detection).
MFA	Multi-Factor Authentication — requires two or more independent factors (knowledge + possession + inheritance).
DPDP	Digital Personal Data Protection Act, 2023 — India's data protection law governing collection, processing, and storage of personal data.
PITR	Point-In-Time Recovery — PostgreSQL feature allowing restoration to any second within retention window via WAL archive.
SIEM	Security Information and Event Management — centralized log aggregation + correlation + alerting platform.
SAST / DAST	Static / Dynamic Application Security Testing — source-code vs. running-app security scanning.
BOLA / IDOR	Broken Object Level Authorization / Insecure Direct Object Reference — vulnerability where user can access another user's objects by ID.
CSP	Content-Security-Policy — HTTP response header controlling resource origins the browser is allowed to load.
HSTS	HTTP Strict-Transport-Security — header forcing browsers to use HTTPS for the specified domain.
SSE-KMS	Server-Side Encryption with AWS KMS-managed keys — S3 encryption mode using customer-managed keys.
KMS	Key Management Service — AWS service for creating + managing cryptographic keys with IAM-scoped access.